

Action-Linked Credit Assignment for Event-Driven Warehouse Management via Multi-Agentic Reinforcement Learning

Abstract

E-commerce fulfillment warehouses face increasing operational complexity due to rapid order growth and evolving product assortments. Traditional rule-based approaches for warehouse space allocation and inventory management struggle to adapt to such dynamic environments, often resulting in suboptimal performance. We propose an integrated reinforcement learning (RL)-based framework that jointly optimizes warehouse slotting and inventory replenishment through an event-driven architecture, enabling real-time responsiveness to operational changes. The system comprises a Slotting Agent (*SA*) for zone assignment and an Inventory Agent (*IA*) for replenishment decisions. Crucially, the *IA* introduces a delayed reward mechanism that explicitly links replenishment decisions to realized inventory outcomes after lead times, allowing the agent to learn account for the temporal gap between ordering actions and their effects for balanced warehouse utilization and effective inventory control. Furthermore, embedding-based similarity matching enables the learned policies to generalize to new products without retraining, addressing a key limitation of existing RL approaches. Experimental results show that the proposed *SA+IA* framework achieves over ten times higher cumulative reward than rule-based baselines, reducing replenishment orders by 19% and stockout ratios by 35%. In dynamic environments with new product introductions, the framework maintains stable performance while baseline methods experience severe degradation. Ablation studies further confirm that without delayed rewards, the agent cannot learn the long-term impact of ordering decisions under lead time uncertainty, leading to higher stockout ratios. Overall, this work demonstrates the practical applicability of RL-based decision-making for automated warehouse operations in real-world e-commerce environments.

1. Introduction

With the rapid expansion of e-commerce, the logistics paradigm has shifted from large-scale batch operations to small-batch, multi-variety and rapid delivery systems (Fonseca-Galindo et al., 2022; Trindade et al., 2022; Wan et al., 2021). Thus, fulfillment logistics systems have evolved into integrated subsystems that manage and connect all processes from inbound to outbound in real time – including warehousing, order processing, and shipment – extending well beyond their traditional storage-oriented functions (Gu et al., 2007; Leung, 2025).

As these systems grow in scale and scope, the increasing volume of orders and the demand for same-days delivery have substantially increased the complexity of critical operational decisions, including space utilization, product slotting, and inventory replenishment. Recent advances in automation, artificial intelligence, and data-driven decision-making have made it possible to manage this complexity in a more responsive and efficient manner (Ferreira & Reis, 2023; Nitsche, 2021; Zhen & Li, 2022). Yet, these approaches largely depend on pre-learned patterns and historical data, making it difficult to cope with fluctuations in uncertain demand patterns and to keep pace with changing operational conditions.

Reinforcement Learning (RL) has emerged as a promising approach for logistics operations, capable of autonomously learning optimal policies that maximize long-term rewards through continuous interaction with dynamic environments (Khankhour et al., 2025; Mahmoudnazlou et al., 2025). Nevertheless, existing RL-based warehouse automation research exhibits several limitations that hinder practical deployment in real-world e-commerce settings. First, most existing approaches assume a fixed product set throughout the operation period, requiring complete model restructuring and retraining whenever the product assortment changes (Barat et al., 2019; Cestero et al., 2022; Zoubair et al., 2025). This static assumption contradicts the dynamic nature of e-commerce environments where products are continuously added, removed, or replaced based on market demands and seasonal trends. Second, periodic decision-making frameworks that operate at fixed time intervals, such as daily or weekly batches (Khirwar et al., 2023; Mutyala & Kaur, 2024), cannot promptly respond to real-time environmental changes. This temporal rigidity prevents the system from adapting to sudden demand fluctuations or urgent inventory needs that frequently occur in modern e-commerce operations. Finally, inventory replenishment decisions are subject to lead times spanning multiple days between order placement and stock arrival (Setiawan & Nurhasanah, 2022). In practical logistics environments where lead times range from several days to weeks, current replenishment decisions directly impact future inventory states and operational performance. However, existing approaches fail to explicitly capture these delay effects caused by lead time, resulting in policies that cannot properly attribute delayed outcomes to their originating decisions, leading to suboptimal inventory balance and operational efficiency.

We propose an integrated RL-based decision-making framework for warehouse space optimization and inventory management that is designed to handle the dynamic and real-time e-commerce fulfillment operations. This framework comprises two specialized agents: a Slotting Agent (*SA*) that determines optimal zone allocations for inbound products, and an Inventory Agent (*IA*) that decides replenishment timing and quantity. Both agents employ an event-driven architecture to enable real-time responsiveness to environmental changes and represent states based on individual product units rather than fixed product sets. For new products lacking of historical data, the framework identifies similar existing products via embedding-based similarity and leverages their historical information to fill the missing entries in the new product's state representation, allowing the learned policies to be applied without additional retraining. The *SA* enhances space utilization and picking efficiency by adopting a zone-based spatial approach, whereas the *IA* introduces a Delayed Reward mechanism that explicitly models post-lead-time rewards to maximize long-term inventory efficiency. The proposed framework provides a practical foundation for RL-based warehouse automation in dynamic e-commerce environments.

Experimental results demonstrate that the proposed framework consistently outperforms rule-based baselines in space utilization balance, replenishment efficiency, and stockout reduction, while maintaining stable and robust performance under diverse stochastic operational conditions. Moreover, the framework effectively generalizes to dynamic product compositions with new product introductions during operation, preserving operational stability without additional retraining.

2. Related Work

Warehouse operations and inventory management have traditionally relied on well-established methodologies such as ABC analysis-based slotting policies and Economic Order Quantity (EOQ) models combined with Re-Order Point (ROP) strategies. ABC analysis categorizes products by turnover rate and assigns high-frequency items to accessible locations, with numerous studies validating its effectiveness in improving picking efficiency (Kučera & Dastych, 2018; Kučera & Suk, 2019; Umry & Singgih, 2019). EOQ and ROP models likewise offer a theoretical basis for achieving a balance between ordering costs and inventory holding costs (Cheng et al., 2015; Kumar, 2016; Nobil et al., 2020). However, these conventional techniques represent static models predicated on simplified cost structures and stable demand patterns, rendering them insufficient for comprehensively addressing the complex realities of actual warehouse operations, including demand volatility, lead time uncertainty, and high-variety, low-volume procurement characteristics (Alnahhal et al., 2024; Mojumder & Nuruzzaman, 2025; Rajnikant & Khanna, 2025).

RL has emerged as a compelling alternative, owing to its capacity to learn adaptive policies through

direct interaction with dynamic environments. In the slotting domain, a Deep Q-Network (DQN) based policy has been proposed that allocates products to predefined storage zones in response to real-time order fluctuations, demonstrating that RL can handle dynamic slotting problems beyond the reach of static ABC policies (Waubert de Puiseau et al., 2022). The storage location assignment problem has been reformulated as a sequential decision-making task, applying Double Q-learning to determine rack assignments for restocking arrivals (Troch et al., 2023). In parallel, standardized simulation environment has been developed to facilitate RL-based slotting research under more realistic operational conditions that reflect diverse warehouse configurations and real warehouse processes (Leon et al., 2023).

In the inventory management domain, RL-based methods have shown strong potential for autonomous stock control under demand uncertainty. A Proximal Policy Optimization (PPO)-based single-agent policy has been developed across a factory-warehouse-retailer network, achieving higher profit than optimized base-stock policies on real-world data (Kumabe et al., 2022). A cooperative multi-agent RL framework has been proposed, modeling a warehouse and multiple retail stores as individual agents that jointly minimize supply chain costs, thereby reducing stockouts and excess inventory (Khirwar et al., 2023). RL-based dynamic replenishment models have been proposed for multi-echelon supply chains, coordinating inventory across hierarchical layers using SAC-AlphaLR for continuous control and validated on Fast-Moving Consumer Goods data (Zhang et al., 2024). Nevertheless, RL-based inventory management approaches still retain structural limitations that hinder real-world deployment, including the exponential growth of state and action spaces as SKU counts increase, the need for retraining when product assortments change, the limited real-time responsiveness of periodic decision-making schemes, and the insufficient handling of delayed rewards arising from lead times.

Taken together, prior work in slotting and inventory management has made substantial progress but remains limited by treating these decision processes in isolation, insufficiently accommodating dynamic product assortments and real-time operational requirements, and failing to account for the temporal gap between replenishment decisions and their realized inventory outcomes. The present work integrates *SA* and *IA* within a unified simulation environment, adopts product-level state representations with embedding-based similarity to enable policy generalization to unseen products, and introduces a delayed reward mechanism that explicitly captures post-lead-time inventory consequences. In doing so, the proposed framework provides a more realistic and integrated decision-making approach that reflects the operational complexity of modern e-commerce fulfillment warehouses.

3. Proposed Method

This section presents the structure and design of the proposed RL-based warehouse decision-making

framework. The system is implemented within a simulation environment that reflects the workflow of an e-commerce fulfillment warehouse and automates two critical processes: product slotting and inventory management. Fig. 1 illustrates the overall architecture of the proposed system.

The warehouse environment is modeled using a zone-based structure to consider differences in distance and accessibility from the packing point, while accounting for inventory state changes and spatial constraints arising from inbound and outbound events. It adopts an event-driven architecture, in which state updates occur only when inbound or outbound events take place. These events are generated using probabilistic models based on the statistical characteristics of real operational data, and the corresponding decision-making module is triggered depending on the event type. Upon an inbound event, the *SA* determines the placement location of the product, while upon an outbound event, the *IA* determines the order quantity. The two agents are trained independently in separate simulation environments, with *SA* and *IA* each learning their respective policies through isolated training procedures. After training is completed, the learned policies are jointly evaluated within an integrated simulation environment where they share warehouse state information, enabling assessment of integrated system performance. This approach allows each agent to specialize in its own decision-making task while enabling coordinated behavior through shared environmental interactions during the evaluation phase.

3.1 Slotting Agent Design

The *SA* is responsible for determining the storage zone assignment of products arriving, aiming to improve long-term warehouse space utilization and picking efficiency. The warehouse storage space is divided into four zones – Golden zone, Zone1, Zone2, Zone3 – based on their distance from the packing point. The agent makes slotting decisions by comprehensively considering both the current state of each zone and the characteristics of incoming products. Specifically, the slotting strategy is designed to prioritize placing products with higher turnover rates in zones closer to the packing point, particularly the Golden zone, while maintaining balanced space utilization across zones to prevent congestion or inefficient placement.

3.1.1 State Representation

The state of the *SA* consists of spatial information for each zone and the features of the incoming products. The state at time t , denoted as s_t^{SA} , is defined Eq. (1) as follows:

$$s_t^{SA} = (z_t^{remain}, z_t^{usage}, z_t^{turnover}, p_t^{weight}, p_t^{turnover}) \#(1)$$

Here, $z_t^{remain} \in \mathbb{R}^4$ represents the remaining capacity ratio of each of 4 zones, $z_t^{usage} \in \mathbb{R}^4$ denotes the proportion of the total warehouse weight currently utilized by each zone, and $z_t^{turnover} \in \mathbb{R}^4$ indicates the quantity-weighted average turnover rate of products stored in each zone. The features of

the incoming product are represented by $p_t^{weight}, p_t^{turnover}$, indicating the product's weight and turnover rate, respectively.

The product turnover rate is calculated based on outbound history using a time-weighted approach given by Eq. (2):

$$\text{Turnover Rate}_i = \frac{\sum_k q_k}{\sum_k (t_k^{out} - t_k^{in}) \cdot q_k} \quad \#(2)$$

where k indexes each outbound event for product i , q_k is the quantity shipped, t_k^{out} is the outbound timestamp, and t_k^{in} is the corresponding inbound timestamp. This metric indicates how quickly inventory is depleted, with higher values representing faster turnover. Each time an outbound event occurs, the stored turnover rate is updated by averaging the previous value and the newly calculated rate.

3.1.2 Action Space

The action of the SA is defined in Eq. (3) as selection of a storage zone for incoming products:

$$a_t^{SA} \in \{\text{Golden zone}, \text{Zone1}, \text{Zone2}, \text{Zone3}\} \quad \#(3)$$

If the remaining capacity of the selected zone is insufficient to accommodate the weight of the incoming product, the environment enforces a capacity constraint by placing the product in a zone with available capacity. The penalty for this capacity violation is reflected in the reward function.

3.1.3 Reward Function

The reward function for the SA is designed to incorporate balanced space utilization, turnover-based placement suitability, and penalties for capacity violations as expressed in Eq. (4):

$$r_t^{SA} = R_{balance} + R_{match} + R_{overweight} \quad \#(4)$$

Each component of the reward function is defined as follows.

First, the space utilization balance reward $R_{balance}$ encourages minimizing the variance of space utilization across zones according to Eq. (5):

$$R_{balance} = -\frac{1}{N} \sum_{i=1}^N (\text{Usage Ratio}_{\text{Zone}_i} - \mu)^2 \quad \#(5)$$

where N denotes the number of zones, $\text{Usage Ratio}_{\text{Zone}_i}$ is the utilization rate of zone i , and μ represents the average utilization rate across all zones. This term assigns higher rewards when space utilization is evenly distributed across zones.

Second, the turnover-based placement reward R_{match} evaluates the suitability of the selected zone with respect to the product's turnover rate. To construct zone-specific turnover intervals, products currently stored in the warehouse are sorted in descending order of their turnover rates. The sorted list is then partitioned according to the storage capacity ratio of each zone, and the turnover values at the partition boundaries define the intervals $Q_{Zone} \in \{Q_{Golden}, Q_{Zone1}, Q_{Zone2}, Q_{Zone3}\}$, each corresponding to a specific zone. The ideal zone Z_{ideal} is defined as the zone whose turnover interval Q_{zone} contains the product's turnover rate $p_t^{turnover}$, while $Z_{selected}$ denotes the zone chosen by the *SA*. The zone distance d is computed as the absolute difference between the indices of the two zones, reflecting how far the selected zone deviates from the ideal one within the four-zone warehouse structure. Based on this distance, R_{match} is assigned as shown in Eq. (6):

$$R_{match} = \begin{cases} +1.0, & \text{if } d = 0 \\ +0.5, & \text{if } d = 1 \\ -0.5, & \text{if } d = 2 \\ -1.0, & \text{if } d = 3 \end{cases} \#(6)$$

where $d = |idx(Z_{ideal}) - idx(Z_{selected})|$, and $idx(\cdot)$ returns the zone index (0 for Golden zone, 1 for Zone1, 2 for Zone2, 3 for Zone3). This relative evaluation scheme assigns higher rewards to placements closer to the ideal zone and larger penalties as the deviation increases, allowing the *SA* to determine placement priorities by comparing incoming products with those currently stored in the warehouse.

Third, the overweight penalty $R_{overweight}$ penalizes decisions that exceed the capacity of the selected zone by imposing a negative reward proportional to the amount by which the zone's total weight surpasses its maximum allowable capacity as defined in Eq. (7):

$$R_{overweight} = -\max(0, w_{after} - w_{max}) \#(7)$$

where w_{after} denotes the total weight of the zone after placing the product, and w_{max} represents the maximum allowable weight of the zone.

3.2 Inventory Agent Design

The *IA* determines the order quantity for products when an outbound event occurs, with a zero-quantity indicating no order. The primary objective of the *IA* is to minimize long-term inventory operating costs by dynamically adjusting inventory levels under demand volatility and uncertainty, thereby reducing stockouts and mitigating the risk of surplus inventory. Notably, the *IA*'s reward structure comprises an immediate reward evaluated at the time of ordering and a delayed reward assigned when the ordered products are actually received. This design allows the agent to account for post-inbound changes in inventory dynamics when making ordering decisions.

3.2.1 State Representation

The state of the *IA* includes information on product inventory and physical characteristics, future demand forecasts, and the warehouse operating environment. At time t , the state s_t^{IA} is defined in Eq. (8) as follows:

$$s_t^{IA} = (x_t^{prod}, x_t^{demand}, x_t^{warehouse}) \#(8)$$

Here, x_t^{prod} represents product-level features, including current inventory level, product weight, box weight, turnover rate and lead time. The term x_t^{demand} denotes demand-related information and consists of recent demand time series data, forecasted daily demand and inventory shortfall relative to target inventory levels (See Supplementary Material 3 for details on the demand forecasting model). Finally, $x_t^{warehouse}$ encompasses the warehouse-level information, such as space utilization, remaining storage capacity, and expected inbound and outbound weights.

3.2.2 Action Space

In the warehouse operating environment, inbound processes are handled in box units, whereas outbound processes are conducted at the individual item level. Consequently, the *IA* determines the order quantity in box units. The agent outputs a continuous action variable $a_t \in [-1, 1]$, which is linearly transformed into an order ratio $r_t \in [0, 1]$ using Eq. (9):

$$r_t = \frac{a_t + 1}{2} \#(9)$$

The actual order quantity Q_t^{order} at time t is computed according to Eq. (10) by incorporating order unit constraints:

$$Q_t^{order} = \max\left(0, \left\lfloor \frac{r_t \cdot Q_i^{max}}{n_i^{unit}} \right\rfloor \cdot n_i^{unit}\right) \#(10)$$

Where Q_i^{max} denotes the maximum loadable quantity of product i in the warehouse, and n_i^{unit} represents the order unit size. To prevent duplicate ordering, further orders are restricted if a previously placed order for the same product has not yet been received into inventory. This constraint mitigates unnecessary repeated orders and reduces the risk of excess inventory when inventory updates are delayed due to lead times.

3.2.3 Reward Function with Delayed reward

The *IA* employs a reward structure composed of immediate and delayed rewards to reflect both the immediate impact of ordering decisions and the delayed inventory outcomes caused by lead time, guiding the *IA* to learn policies that balance short-term responsiveness with long-term inventory

performance. The total reward for the IA at time t , denoted as r_t^{IA} , is defined in Eq. (11):

$$r_t^{IA} = R_t^{imm} + R_{t+\Delta L}^{del} \#(11)$$

Here, R_t^{imm} is the immediate reward computed at the order decision time, and $R_{t+\Delta L}^{del}$ is the delayed reward calculated at the inbound time $t + \Delta L$. The $R_{t+\Delta L}^{del}$ is retrospectively attributed to the state-action pair associated with the ordering decision at time t . To support this mechanism, the environment tracks both the order id and the expected inbound time. Once the inbound process is completed, the $R_{t+\Delta L}^{del}$ is combined with the R_t^{imm} to construct the final reward r_t^{IA} . The propagation process of delayed reward $R_{t+\Delta L}^{del}$ is illustrated in Fig 2.

The immediate reward captures the short-term consequences upon order and consists of two components as shown in Eq. (12):

$$R_t^{imm} = r_t^{timing} + r_t^{capacity} \#(12)$$

The order timing reward r_t^{timing} evaluates whether an ordering decision is made too early, resulting in overstocking, or too late, thereby increasing stockout risk. The target inventory level Q_t^{ideal} is defined based on the ROP, and inventory status is evaluated by its deviation from the current inventory level $Q_t^{current}$. A positive deviation indicates an understocked state relative to the target, whereas a negative deviation corresponds to overstocking. Accordingly, the r_t^{timing} is defined in Eq. (13) as:

$$r_t^{timing} = s_t \cdot (|Q_t^{ideal} - Q_t^{current}|) \#(13)$$

Here, $s_t \in \{+1, -1\}$ denotes a sign term determined by the combination of the inventory state and the ordering action. As a result, reward or penalty for order timing are strengthened as the inventory state deviates further from the target level.

The capacity reward $r_t^{capacity}$ is computed using Eq. (14) based on warehouse utilization at the ordering moment:

$$r_t^{capacity} = -\frac{W_t^{used}}{W_t^{max}} \#(14)$$

where W_t^{used} and W_t^{max} represent the currently utilized warehouse capacity and the maximum available capacity, respectively. This term incentivizes the IA to jointly consider inventory levels and physical storage constraints when making ordering decisions.

The delayed reward is computed at the inbound time $t + \Delta L$, and consists of two components as expressed in Eq. (15):

$$R_{t+\Delta L}^{del} = r_{t+\Delta L}^{inv} + r_{t+\Delta L}^{over} \#(15)$$

To define the inventory level reward $r_{t+\Delta L}^{inv}$, the inventory level observed at the inbound time $t + \Delta L$ is denoted by $Q_{t+\Delta L}^{current}$. The potential inventory level $\tilde{Q}_{t+\Delta L}$ resulting from the ordering decision at time t represents the IA 's intended inventory state without warehouse capacity constraints, calculated by Eq. (16):

$$\tilde{Q}_{t+\Delta L} = Q_{t+\Delta L}^{current} + Q_t^{order} \#(16)$$

The inventory upper bound Q^{over} is established by incorporating the target inventory level and turnover rate, as expressed in Eq. (17):

$$Q^{upper} = Q_t^{order} \cdot (1 + p_{t+\Delta L}^{turnover}) \#(17)$$

The inventory level reward $r_{t+\Delta L}^{inv}$ is assigned according to Eq. (18) based on the degree to which the potential inventory level deviates from the target range:

$$r_{t+\Delta L}^{inv} = \begin{cases} -(\max(0, Q_t^{ideal} - \tilde{Q}_{t+\Delta L}, \tilde{Q}_{t+\Delta L} - Q^{upper})), & \text{if } Q_t^{order} > 0 \\ 0.0, & \text{if } Q_t^{order} = 0 \end{cases} \#(18)$$

Under this formulation, a penalty is imposed for stockout risk when the inventory level falls below the target inventory level Q_t^{ideal} , whereas excessive inventory beyond the defined upper bound Q^{upper} incurs a penalty. However, when no order is placed ($Q_t^{order} = 0$), excess inventory is not attributed to the IA 's decision and therefore no inventory level reward is assigned.

In practical operational environments, capacity constraints may prevent full realization of the IA 's intended inventory level. Accordingly, a separate warehouse overflow penalty $r_{t+\Delta L}^{over}$ is imposed on the incoming quantity that cannot be physically accommodated in the warehouse, as defined in Eq. (19):

$$r_{t+\Delta L}^{over} = -(\tilde{Q}_{t+\Delta L} - Q_{t+\Delta L}^{accepted}) \#(19)$$

where $Q_{t+\Delta L}^{accepted}$ denotes the quantity actually accepted into the warehouse.

3.3 Adaptation to Dynamic Product Composition

In e-commerce fulfillment environments, product assortments evolve continuously through new product launches, discontinuations, and seasonal replacements. Consequently, RL-based warehouse operation systems trained on fixed product sets can readily confront the limitations in real-world applicability. The SA and IA in this study are designed to operate at the product level, enabling the learned policy to be applied to new products without requiring agent retraining. However, newly introduced products lack the historical data - including demand patterns, turnover rates, and forecast information- necessary for state construction. To address this issue, the proposed framework employs an embedding model that maps the new product into a shared embedding space alongside existing

products, based on attributes such as category, color, brand, and functional features. Subsequently, a reference product that is most similar to the new product is identified based on cosine similarity between the embedding vectors. Fig. 3(c) visualizes the product embedding space using t-SNE, showing that products form clusters by brand and category. New products (black asterisks) are projected into this space and assigned reference products by finding the most similar existing products within the corresponding clusters. The *SA* uses the turnover rate of the reference product for zone assignment decisions, while the *IA* supplements the state representation of the new product by borrowing the turnover rate, historical demand, and forecast demand.

4. Experiments

This section evaluates the proposed RL-based warehouse operation framework through comprehensive experiments. We first describe the experimental setup, then compare the proposed approach with baseline methods, and finally analyze the contribution of key design components through ablation studies.

4.1 Simulation Environment and Data Generation

We develop an event-driven simulation environment that reflects the real-time operational characteristics and dynamic product composition of e-commerce fulfillment warehouses. The simulation operates at a minute-level time scale and triggers decision-making only upon inbound or outbound events. The environment consists of 10 products and a warehouse divided into four zones based on distance and accessibility from the packing point, with predefined capacity ratios assigned to each zone.

Due to limited access to real-world operational data, the simulation dataset is generated using statistical characteristics extracted from a publicly available logistics dataset (*Mega Star Distribution Centre Dataset*, n.d.). Event occurrences follow probabilistic models based on product-specific frequencies, with processing quantities sampled from Poisson distributions. The *SA* training dataset simulates concurrent inbound and outbound events (Fig. 3(a)), while the *IA* dataset incorporates only outbound events with category-level trends and brand-level promotional effects to capture demand volatility (Fig. 3(b)). Order lead times are sampled from category-specific ranges up to five days, with a minimum of one day enforced. For the *IA*, each simulation episode spans 120 days, with the first 60 days used as a warm-up period to construct demand history and forecasting inputs, and the remaining period used for training and evaluation. Additional implementation details are provided in the Supplementary Material 2.

4.2 Baseline Methods

To evaluate the performance of the proposed framework, we select rule-based decision policies widely adopted in practical warehouse operations as baselines. All baseline policies are evaluated in the same simulation environment as the proposed method, using identical inbound and outbound event streams, lead time settings, and operational constraints.

For the slotting baseline, we adopt an ABC slotting policy based on class-based storage (McInerney & Yadavalli, 2022). Products are ranked in descending order by their observed outbound frequency during the warm-up period. Rank boundaries are then determined according to the predefined capacity ratios of the storage zones, and products are assigned to zones accordingly. This procedure extends the traditional class-based storage approach, in which zone sizes and boundaries are defined first and products are allocated in order of operational importance.

As the inventory baseline, we employ an inventory control model based on ROP and Safety Stock, where replenishment is triggered when the inventory level falls below a reorder point s , restoring inventory to a target level S (Pamolango & Dewantoro, 2025). We adopt a demand-based parameterization as shown in Eqs. (20)-(21):

$$s = \mu_D \cdot L \quad (20)$$

$$S = \mu_D \cdot (L + R) + z_{0.95} \cdot \sigma_D \cdot \sqrt{L} \quad (21)$$

Where μ_D and σ_D denote the mean and standard deviation of daily demand, L is the lead time, R is the review period, representing the interval at which inventory levels are reviewed (set equal to L), and $z_{0.95}$ is the z-score corresponding to a 95% service level. s represents the expected demand during lead time, while S comprises the expected demand over the replenishment cycle (first term) and safety stock (second term).

The same ordering constraint as the proposed RL-based method is applied, restricting additional orders when previously placed orders have not yet arrived at the warehouse.

4.3 Main Results

In this section, we analyze the performance of the proposed RL-based warehouse operation framework under two experimental scenarios. To demonstrate the robustness of the learned policies, we conduct all experiments over multiple independent episodes with different random seeds, generating diverse realizations of demand patterns and event timing. First, we evaluate the integrated performance under conditions identical to the training environment. Then, we verify the generalization capability and operational stability of the proposed method in a dynamic product composition environment where new products are introduced during operation. Hereafter, we denote the proposed RL-based integrated framework as *SA+IA* and the rule-based baseline method as *Ruled*.

4.3.1 Performance Comparison in Static Product Environment

We first analyze the integrated performance of the proposed framework in a standard evaluation environment identical to the training setup.

Fig. 4(a) compares cumulative total reward trajectories over the evaluation period, which reflect overall warehouse operational performance. The shaded regions represent the standard deviation across multiple evaluation episodes. *SA+IA* achieves over ten times higher cumulative reward than *Ruled*, with consistently superior performance across all episodes. Moreover, the narrow shaded region for *SA+IA* signifies low performance variability and confirms that the proposed RL-based framework delivers stable and reproducible behavior under diverse stochastic operational scenarios, thereby demonstrating the robustness of the learned decision-making policies. In contrast, *Ruled* frequently experiences prolonged negative returns, highlighting its sensitivity to stochastic demand and event timing. Space utilization performance is presented in Fig. 4(b), showing the daily difference between the maximum and minimum zone utilization across the four zones over time. *SA+IA* consistently maintains substantially lower zone imbalance compared to *Ruled* throughout the evaluation period, effectively mitigating excessive concentration of storage usage and demonstrating improved balance in space allocation across zones. The stable performance pattern across episodes suggests that the *SA* learns adaptive placement policies by accounting for current zone utilization, thereby preventing congestion in specific zones even under varying stochastic demand and event-timing conditions. Figs. 4(c) and 4(d) jointly demonstrate the superior inventory management performance of *SA+IA*. As shown in Fig. 4(c), *SA+IA* places an average of 17.75 replenishment orders per product, approximately 19% fewer than *Ruled*'s 21.38 orders. Despite fewer ordering actions, *SA+IA* achieves superior service levels, as evidenced in Fig. 4(d) by a substantially lower mean stockout ratio of 6.77% compared to 10.48% for *Ruled* - a reduction of approximately 35%. Moreover, the stockout distribution of *SA+IA* is notably tighter, indicating effective mitigation of extreme stockout risks. These results reveal that *SA+IA* reduces ordering frequency without sacrificing inventory availability, confirming that the *IA* learns effective replenishment policies by accounting for lead times and demand variability, thereby eliminating unnecessary orders while maintaining stable service performance across diverse stochastic operational conditions. Overall, these results validate the robustness and practical effectiveness of the proposed *SA+IA* framework for warehouse operations.

4.3.2 Generalization Performance under Dynamic Product Introductions

In the second experiment, we evaluate the generalization performance of the proposed framework under a dynamic product composition environment, reflecting real-world e-commerce fulfillment warehouse settings where product assortments continuously evolve during operation. The evaluation starts with eight products used during training, and two new products not used during training are introduced at

different time points during evaluation, as indicated by the green dashed lines in Fig. 5(a). When new products appear in the simulation environment, reference products are selected using the embedding-based similarity matching method described in Section 3.3, and the state information is supplemented accordingly. For comparison, we denote the variant that applies embedding-based new product mapping to *Ruled* as *Ruled+Emb*.

Fig. 5(a) depicts the cumulative total reward trajectories in the presence of new product inflow, with shaded regions representing the standard deviation across multiple episodes. Notably, while *Ruled* and *Ruled+Emb* display clear performance disruptions following new product introductions, *SA+IA* maintains a consistently increasing reward trajectory throughout the evaluation period, demonstrating robust adaptation across all episodes. *Ruled* undergoes cumulative rewards declining into negative territory and large standard deviations across episodes, indicating unstable behavior that prevents reliable and consistent warehouse operations under dynamic product introductions. *Ruled+Emb* partially improves upon *Ruled* through embedding-based similarity matching and shows limited stabilization after product introduction, but its performance still falls markedly short of *SA+IA*. Fig. 5(b) presents space utilization performance, measured as the daily zone utilization imbalance over time. *SA+IA* maintains consistently low and stable imbalance throughout the entire evaluation period with new products introductions neither increasing the imbalance nor causing volatility. In contrast, *Ruled* exhibits persistently higher imbalance with greater fluctuations, signifying sustained uneven space usage. These results confirm that *SA+IA* preserves balanced space utilization even when new products are introduced, effectively avoiding spatial congestion in specific zones. Fig. 5(c) compares stockout ratios using box plots that visualize the distribution across episodes, illustrating results for all products and separately for newly introduced products. For the total product set, the three methods show comparable mean stockout ratios-*SA+IA* at 5.73%, *Ruled* at 5.4%, and *Ruled+Emb* at 6.16%- similar distributions. However, a striking difference emerges when examining new products specifically, as *SA+IA* achieves a remarkably low mean stockout ratio of 0.58% with minimal variance, whereas *Ruled* records 3.17% and *Ruled+Emb* records 3.8%. This result suggests that *IA* effectively infers demand characteristics of new products from similar products through embedding-based state construction and translates this information into stable replenishment decisions from the outset, thereby preventing the elevated stockout risk typically associated with newly introduced products.

Overall, these results demonstrate that the proposed *SA+IA* framework maintains stable and efficient warehouse operations without retraining, even under dynamic product compositions where new products are continuously introduced. The consistent performance across multiple episodes, together with near-zero stockout ratio for new products, confirms that embedding-based similarity matching effectively enables RL-based policies to generalize to products not used during training while preserving operational robustness.

4.4 Ablation Study on the Delayed Reward Mechanism

In this section, we conduct an ablation study to analyze the effectiveness of the delayed reward mechanism introduced in *IA*. This experiment is conducted in the same standard evaluation environment as the training setup, comparing performance against a variant model that excludes the delayed reward (DR) mechanism (*w/o DR*). The *w/o DR* computes the delayed reward terms based on predicted future states at the time of decision and does not incorporate actual observations after the lead time. While this represents a common RL approach, it has limited ability to correctly attribute long-term inventory outcomes to replenishment decisions under lead time uncertainty.

Fig. 6(a) compares cumulative total reward over the evaluation period. *SA+IA* exhibits a steady and sustained increase in cumulative reward, whereas *w/o DR* achieves substantially lower rewards and shows limited improvement over time. This gap indicates that removing delayed rewards prevents the agent from correctly associating replenishment decisions made at order time with the actual inventory and capacity conditions observed after order arrival, resulting in degraded overall warehouse performance. Fig. 6(b) presents daily warehouse space utilization. *SA+IA* consistently operates with sufficient capacity margin below the maximum capacity, while *w/o DR* remains close to capacity for most of the evaluation period. This behavior reflects overly aggressive ordering driven by short-term considerations, without accounting for post-arrival system-wide effects. The impact of this imbalance becomes evident in Fig. 6(c), which compares stockout ratios. *SA+IA* achieves a mean stockout ratio of 6.77%, compared to 10.48% for the *Ruled* and a substantially higher 40.46% for the *w/o DR*. This severe degradation in the *w/o DR* arises because the agent cannot accurately anticipate post-arrival inventory and capacity conditions, leading it to place excessively large orders to avoid short-term stockouts. As a result, inventory accumulates unevenly across products, pushing the warehouse close to capacity limits and constraining the replenishment of other items, which ultimately causes frequent stockouts despite high overall space utilization. In contrast, *SA+IA* equipped with the delayed reward mechanism learns directly from actual inventory states observed after order arrival. By explicitly associating replenishment decisions with their realized long-term outcomes, the agent learns to balance order quantities under capacity constraints, resulting in more stable inventory allocation and significantly reduced stockout risk. These results demonstrate that the delayed reward mechanism is a critical component of effective RL-based inventory management. Incorporating post-arrival observations enables accurate long-term credit assignment under lead time delays, leading to balanced space utilization and substantially improved operational performance.

5. Conclusion

This study proposes an RL-based warehouse operation framework that jointly optimizes space

allocation and inventory replenishment in e-commerce fulfillment warehouse environments. The framework employs an event-driven architecture with zone-based warehouse modeling that accounts for distance-based accessibility and capacity constraints. It comprises a *SA* for zone assignment and an *IA* for replenishment decisions, where the *IA* is equipped with a delayed reward mechanism to learn from actual inventory outcomes observed after lead times. Additionally, by leveraging embedding-based similarity matching to apply learned policies to new products without retraining, the system enables stable operations even under dynamic product composition environments.

Experimental results demonstrate that the proposed *SA+IA* framework consistently outperforms rule-based methods in terms of cumulative reward, space utilization balance, and stockout ratio, while maintaining stable operations without performance degradation even when new products are introduced. The ablation study confirms that the delayed reward mechanism plays a critical role in learning long-term ordering effects under lead time uncertainty. These findings suggest that explicitly modeling post-lead-time consequences is essential for effective RL-based inventory management in real-world settings. Despite these promising results, several directions remain for future research. The framework is validated within a simulated single-warehouse environment and does not yet account for multi-warehouse or multi-echelon supply chain structures. Furthermore, the absence of cost-based objective functions limits direct economic interpretability. Future work will focus on validation with real operational data, extension to more complex warehouse layouts and multi-facility environments, and the integration of cost-aware objectives to enhance the practical applicability of the proposed approach.

By jointly addressing slotting and inventory decisions through a unified RL framework, this work provides a foundation for the design of adaptive, automated warehouse management systems capable of operating under the dynamic and uncertain conditions characteristic of modern e-commerce fulfillment.

Reference

- Alnahhal, M., Aylak, B. L., Al Hazza, M., & Sakhrieh, A. (2024). Economic order quantity: A state-of-the-art in the era of uncertain supply chains. *Sustainability*, 16(14), 5965.
- Barat, S., Khadilkar, H., Meisheri, H., Kulkarni, V., Baniwal, V., Kumar, P., & Gajrani, M. (2019). Actor based simulation for closed loop control of supply chain using reinforcement learning. *Proceedings of the 18th International Conference on Autonomous Agents and Multiagent Systems*, 1802–1804. <https://www.academia.edu/download/79090211/p1802.pdf>
- Cestero, J., Quartulli, M., Metelli, A. M., & Restelli, M. (2022). Storehouse: A reinforcement learning environment for optimizing warehouse management. *2022 International Joint Conference on Neural Networks (IJCNN)*, 1–9. <https://ieeexplore.ieee.org/abstract/document/9891985/>
- Cheng, L., Tsou, C.-S., & Yang, D.-Y. (2015). Cost-service tradeoff analysis of reorder-point-lot-size inventory models. *Journal of Manufacturing Systems*, 37, 217–226.
- Ferreira, B., & Reis, J. (2023). A Systematic Literature Review on the Application of Automation in Logistics. *Logistics*, 7(4). <https://doi.org/10.3390/logistics7040080>
- Fonseca-Galindo, J. C., de Castro Surita, G., Neto, J. M., de Castro, C. L., & Lemos, A. P. (2022). A multi-agent system for solving the Dynamic Capacitated Vehicle Routing Problem with stochastic customers using trajectory data mining. *Expert Systems with Applications*, 195, 116602. <https://doi.org/10.1016/j.eswa.2022.116602>
- Gu, J., Goetschalckx, M., & McGinnis, L. F. (2007). Research on warehouse operation: A comprehensive review. *European Journal of Operational Research*, 177(1), 1–21. <https://doi.org/10.1016/j.ejor.2006.02.025>
- Khankhour, H., Tajani, C., Rafalia, N., & Abouchabaka, J. (2025). Robotic Agents through Scalable Multi-agent Reinforcement Learning for Optimization of Warehouse Logistics. *WSEAS Transactions on Systems and Control*, 20, 42–49.
- Khirwar, M., Gurumoorthy, K. S., Jain, A. A., & Manchenahally, S. (2023). Cooperative Multi-agent Reinforcement Learning for Inventory Management. In G. De Francisci Morales, C. Perlich, N. Ruchansky, N. Kourtellis, E. Baralis, & F. Bonchi (Eds.), *Machine Learning and Knowledge Discovery in Databases: Applied Data Science and Demo Track* (Vol. 14174, pp. 619–634). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-43427-3_37
- Kučera, T., & Dastych, D. (2018). *Use of ABC analysis as management method in the rationalization of logistic warehousing processes: A case study*. <https://dk.upce.cz/items/64f193a4-a45b-4ed3-89d6-98362231fe6d>

Kučera, T., & Suk, A. (2019). *The application of ABC analysis in the logistic warehousing processes*. <https://dk.upce.cz/items/808c3a46-ac7f-4f06-8b41-d69fa12b45e4>

Kumabe, S., Shiroshita, S., Hayashi, T., & Maruyama, S. (2022). *Learning General Inventory Management Policy for Large Supply Chain Network* (arXiv:2204.13378). arXiv. <https://doi.org/10.48550/arXiv.2204.13378>

Kumar, R. (2016). Economic order quantity (EOQ) model. *Global Journal of Finance and Economic Management*, 5(1), 1–5.

Leon, J. F., Li, Y., Martin, X. A., Calvet, L., Panadero, J., & Juan, A. A. (2023). A hybrid simulation and reinforcement learning algorithm for enhancing efficiency in warehouse operations. *Algorithms*, 16(9), 408.

Leung, E. K. H. (2025). Total fulfillment management: Principles, practices and use cases. *Transportation Research Part E: Logistics and Transportation Review*, 194, 103888.

Mahmoudinazlou, S., Sobhanan, A., Charkhgard, H., Eshragh, A., & Dunn, G. (2025). Deep reinforcement learning for dynamic order picking in warehouse operations. *Computers & Operations Research*, 182, 107112. <https://doi.org/10.1016/j.cor.2025.107112>

McInerney, S. E., & Yadavalli, S. (2022). INCREASING WAREHOUSE THROUGHPUT THROUGH THE DEVELOPMENT OF A DYNAMIC CLASS-BASED STORAGE ASSIGNMENT ALGORITHM. *South African Journal of Industrial Engineering*, 32(2). <https://doi.org/10.7166/33-2-2651>

Mega Star Distribution Centre dataset. (n.d.). Retrieved February 6, 2026, from <https://www.kaggle.com/datasets/aikinomichi/mega-star-distribution-centre>

Mojumder, M. U., & Nuruzzaman, M. (2025). AI-Driven Optimization of Warehouse Layout and Material Handling: A Quantitative Study on Efficiency and Space Utilization. *Review of Applied Science and Technology*, 4(02), 233–273.

Mutyala, P. K., & Kaur, A. (2024). Inventory Optimization of Multi-Echelon Supply Chain Under Market Uncertainties. *2024 First International Conference on Pioneering Developments in Computer Science & Digital Technologies (IC2SDT)*, 257–262. <https://ieeexplore.ieee.org/abstract/document/10696052/>

Nitsche, B. (2021). Exploring the potentials of automation in logistics and supply chain management: Paving the way for autonomous supply chains. In *Logistics* (Vol. 5, Issue 3, p. 51). MDPI. <https://www.mdpi.com/2305-6290/5/3/51>

- Nobil, A. H., Sedigh, A. H. A., & Cárdenas-Barrón, L. E. (2020). Reorder point for the EOQ inventory model with imperfect quality items. *Ain Shams Engineering Journal*, 11(4), 1339–1343.
- Pamolango, M., & Dewantoro, A. D. (2025). Determination of Safety Stock and Reorder Point Using Spreadsheet Simulation for Inventory Control of Packaging Materials. *SITEKIN: Jurnal Sains, Teknologi Dan Industri*, 23(1), 187–197. <https://doi.org/10.24014/sitekin.v23i1.38867>
- Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268), 1–8.
- Rajnikant, P. N., & Khanna, R. (2025). SMART EOQ MODELS: INCORPORATING AI AND MACHINE LEARNING FOR INVENTORY OPTIMIZATION. *International Journal of Engineering Science Technologies*, 9(4), 1–27. <https://doi.org/10.29121/ijjoest.v9.i4.2025.709>
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). *Proximal Policy Optimization Algorithms* (arXiv:1707.06347). arXiv. <https://doi.org/10.48550/arXiv.1707.06347>
- Setiawan, D. T., & Nurhasanah. (2022). The Effect of Lead Time toward Supply Control at Paku Payung Club. *Jurnal Ilmu Sosial Politik Dan Humaniora*, 5(2), 11–22. <https://doi.org/10.36624/jisora.v5i2.85>
- Trindade, M. A. M., Sousa, P. S. A., & Moreira, M. R. A. (2022). Ramping up a heuristic procedure for storage location assignment problem with precedence constraints. *Flexible Services and Manufacturing Journal*, 34(3), 646–669. <https://doi.org/10.1007/s10696-021-09423-w>
- Troch, A., Mannens, E., & Mercelis, S. (2023). Solving the Storage Location Assignment Problem Using Reinforcement Learning. *Proceedings of the 2023 8th International Conference on Mathematics and Artificial Intelligence*, 89–95. <https://doi.org/10.1145/3594300.3594314>
- Umry, T. F., & Singgih, M. L. (2019). Inventory management and reorder point (ROP) strategy using ABC analysis methods in textile manufacture. *IPTEK Journal of Proceedings Series*, (5), 1–7.
- Wan, J., Li, X., Dai, H.-N., Kusiak, A., Martínez-García, M., & Li, D. (2021). Artificial-Intelligence-Driven Customized Manufacturing Factory: Key Technologies, Applications, and Challenges. *Proceedings of the IEEE*, 109(4), 377–398. <https://doi.org/10.1109/JPROC.2020.3034808>
- Waubert de Puiseau, C., Nanfack, D. T., Tercan, H., Löbber-Plattfaut, J., & Meisen, T. (2022). Dynamic storage location assignment in warehouses using deep reinforcement learning. *Technologies*, 10(6), 129.
- Zhang, Y., He, L., & Zheng, J. (2024). A Deep Reinforcement Learning-Based Dynamic Replenishment Approach for Multi-Echelon Inventory Considering Cost Optimization. *Electronics*, 14(1), 66.

Zhen, L., & Li, H. (2022). A literature review of smart warehouse operations management. *Frontiers of Engineering Management*, 9(1), 31–55. <https://doi.org/10.1007/s42524-021-0178-9>

Zoubeir, Z., Mohamed, Y., Haouba, A., & Hmetty, S. (2025). *Reinforcement Learning for Optimal Replenishment in Stochastic Assembly Systems*. <https://europepmc.org/article/ppr/ppr1026963>

Caption

Figure 1. System architecture of the proposed RL-based warehouse management framework

The event-driven architecture integrates a Slotting Agent (SA) and an Inventory Agent (IA), where embedding-based similarity enables state construction for unseen products and supports replenishment decisions with delayed reward feedback

Figure 2. Delayed reward mechanism for Inventory Agent training

Outbound events (Product C) generate state-action transitions stored in the rollout buffer, while inbound arrivals (Product B) after the lead time assign delayed rewards that retrospectively update the corresponding transitions

Figure 3. Simulation data characteristics for SA and IA, and product embedding visualization

(a) Concurrent inbound-outbound environment for SA training, showing 6-hour aggregated supply-demand for a representative product (b) Outbound-driven IA environment exhibiting trend and promotional effects on daily demands for a representative product (c) t-SNE embedding visualization revealing clustering by brand and category, with new products (black star, ★) projected via similarity-based reference matching (See Supplementary Fig. S1 for detailed legend)

Figure 4. Performance comparison in the standard evaluation environment

(a) cumulative total reward over the evaluation period (b) Zone utilization imbalance measured as the daily difference between the maximum and minimum zone utilization (c) Number of replenishment orders by product (d) Quantity-based stockout ratio

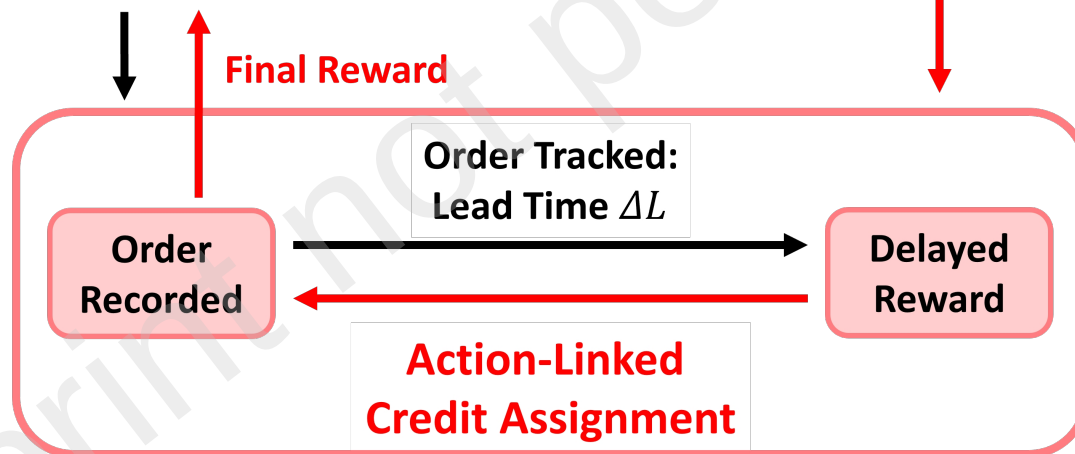
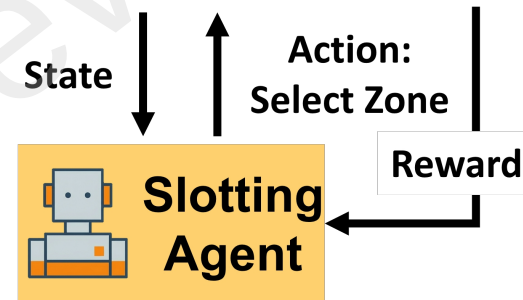
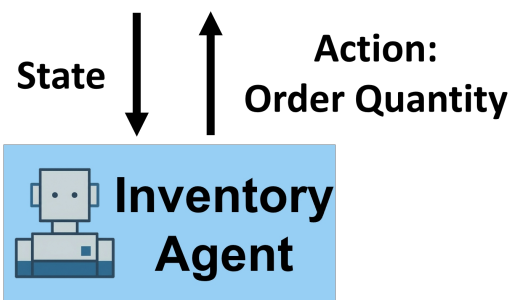
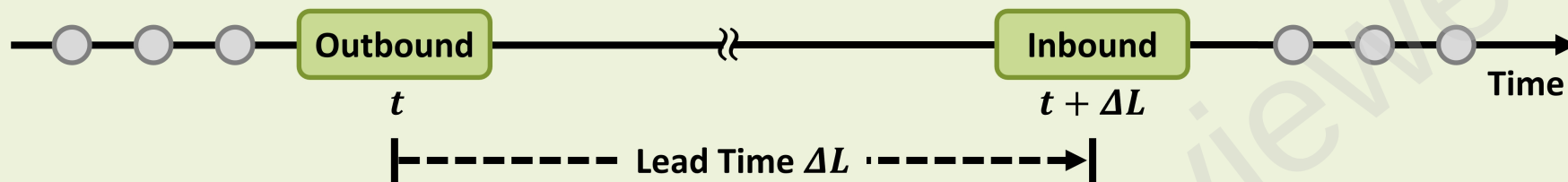
Figure 5. Performance comparison in an environment with new product inflow

(a) Cumulative total reward over time, with green dashed lines indicating the introduction of new products (b) Zone utilization imbalance over time, measured as the daily max-min range across zones (c) Quantity-based stockout ratios for total and only for new products

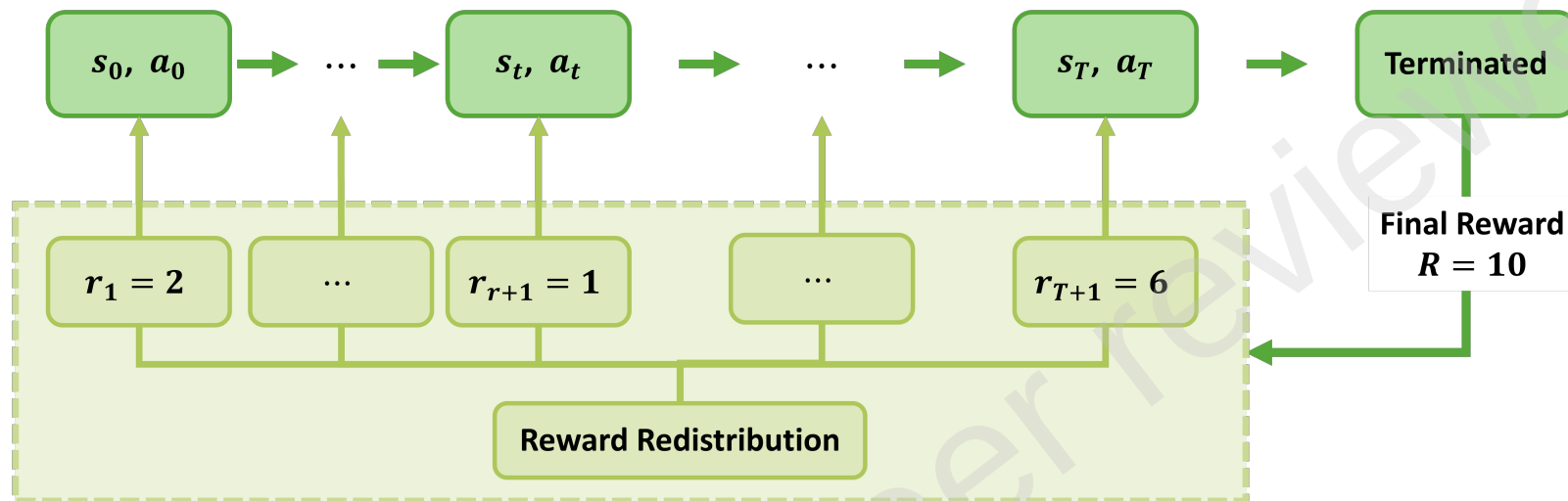
Figure 6. Effect of the delayed reward mechanism on inventory decision performance (a)

Cumulative total reward over the evaluation period (b) Daily warehouse space utilization, with the green dashed line indicating maximum capacity (c) Quantity-based stockout ratio

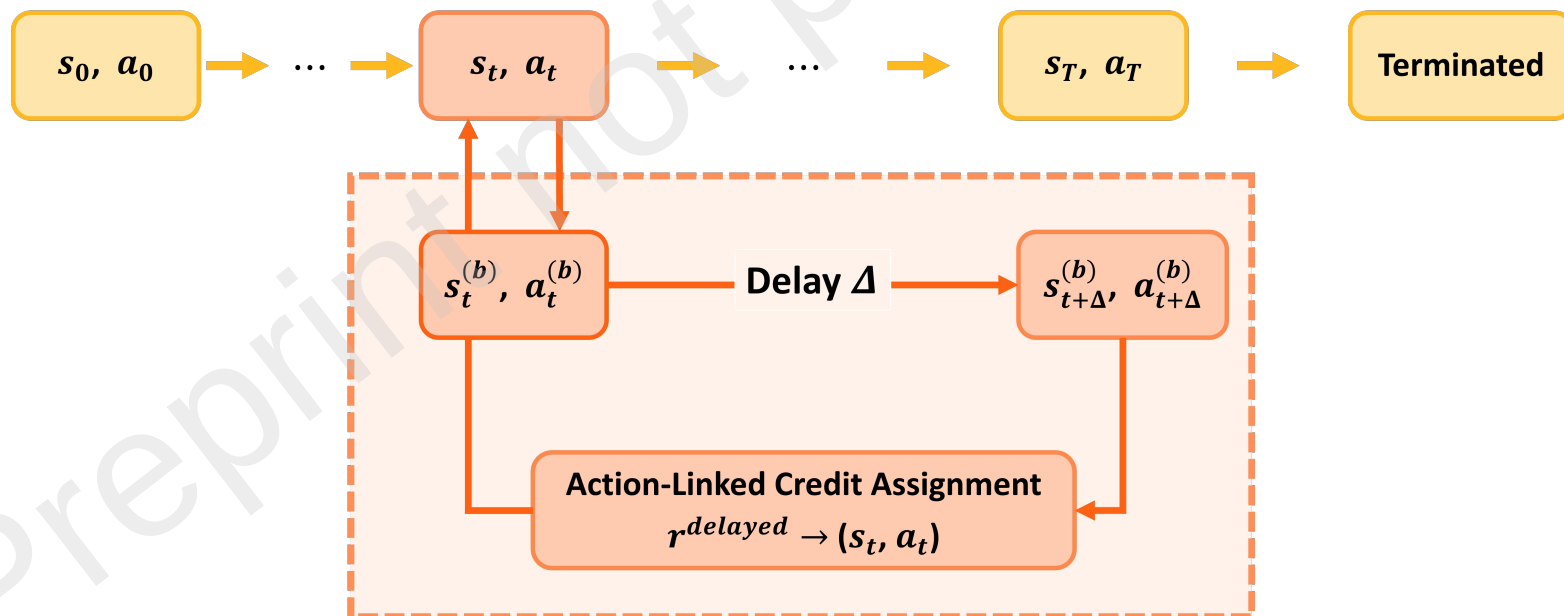
Shared Warehouse Environment

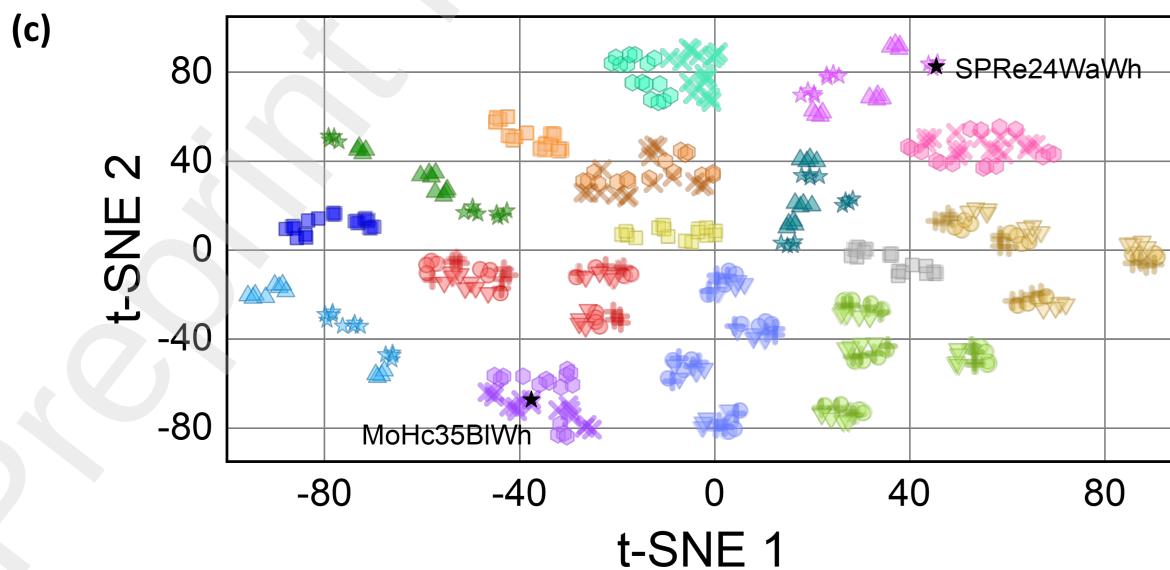
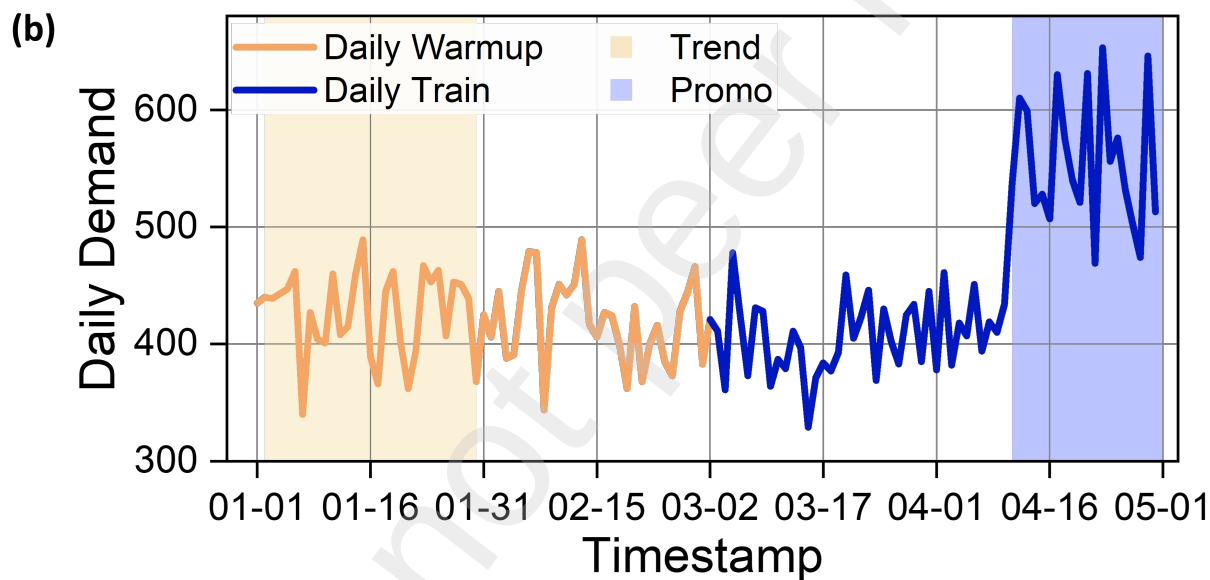
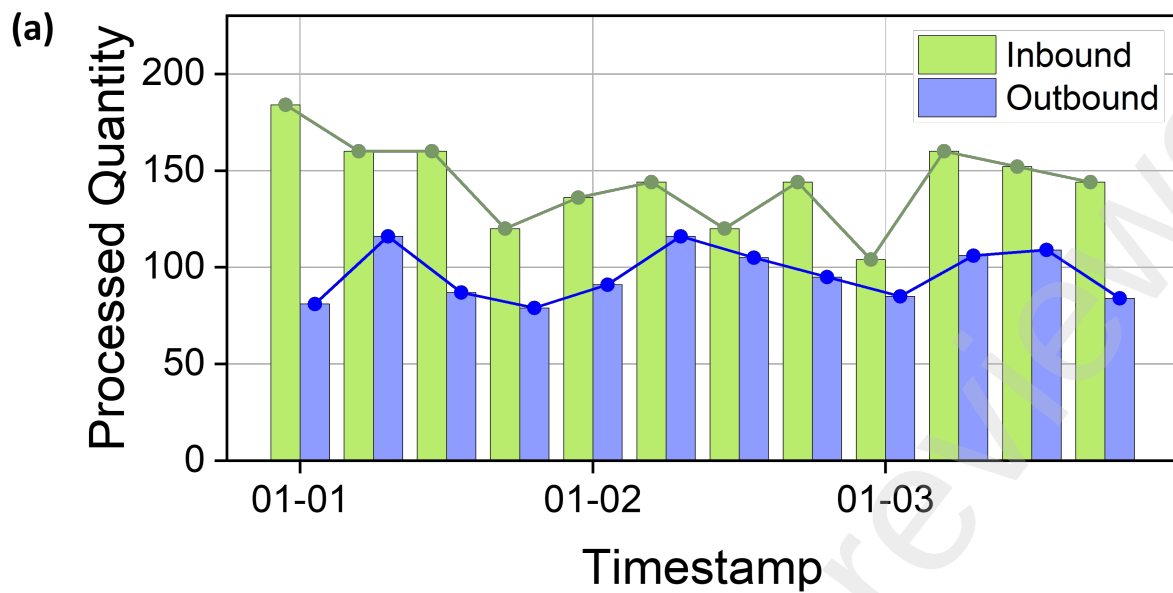


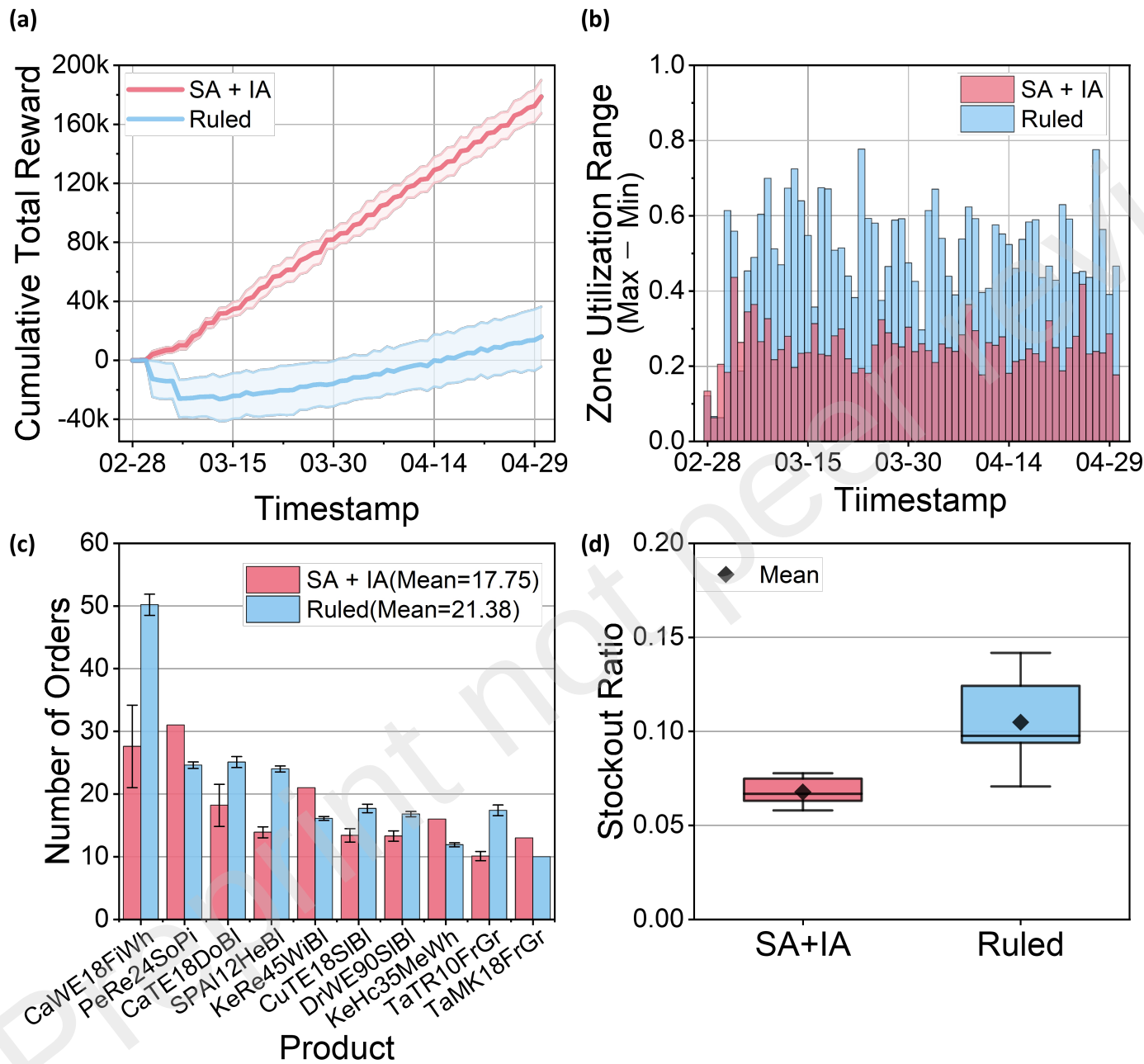
(a) Conventional: Redistributed Delayed Reward (Episode-wise)



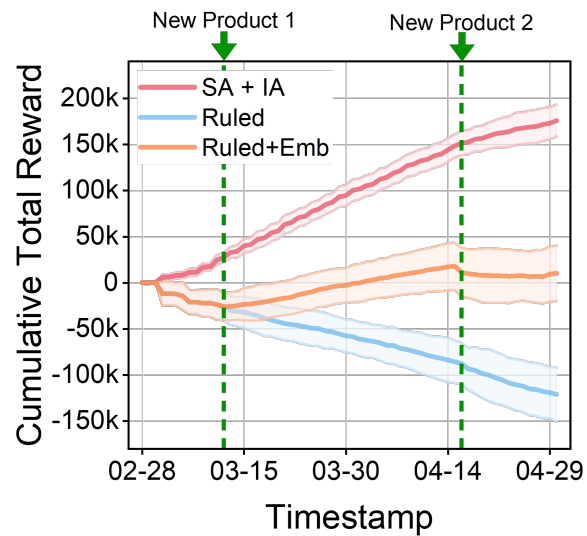
(b) Proposed: Action-Linked Credit Assignment (Action-wise)



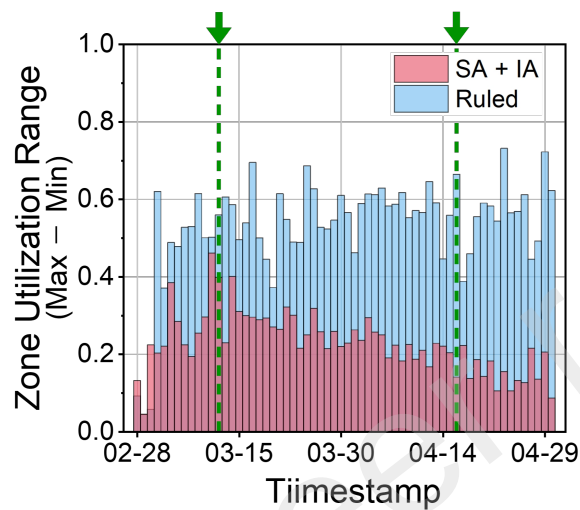




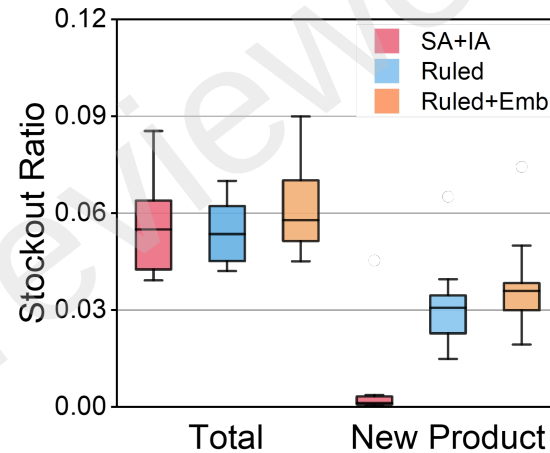
(a)



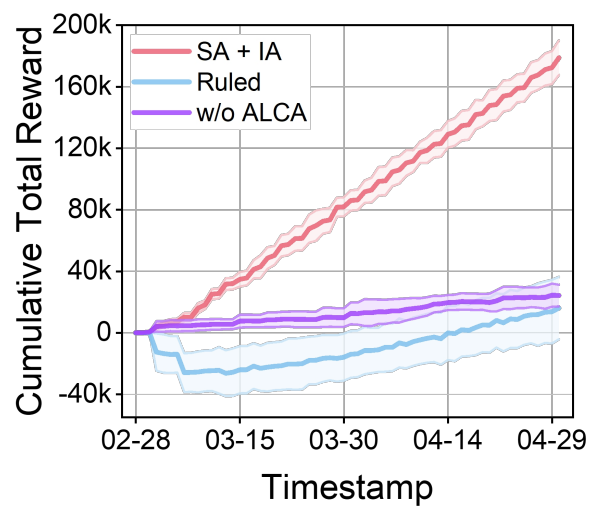
(b)



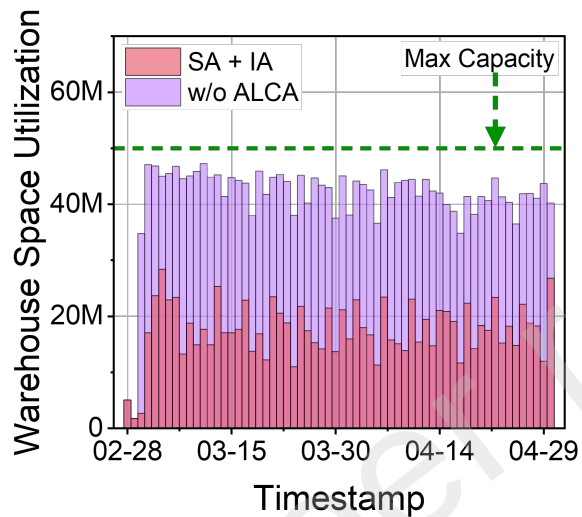
(c)



(a)



(b)



(c)

